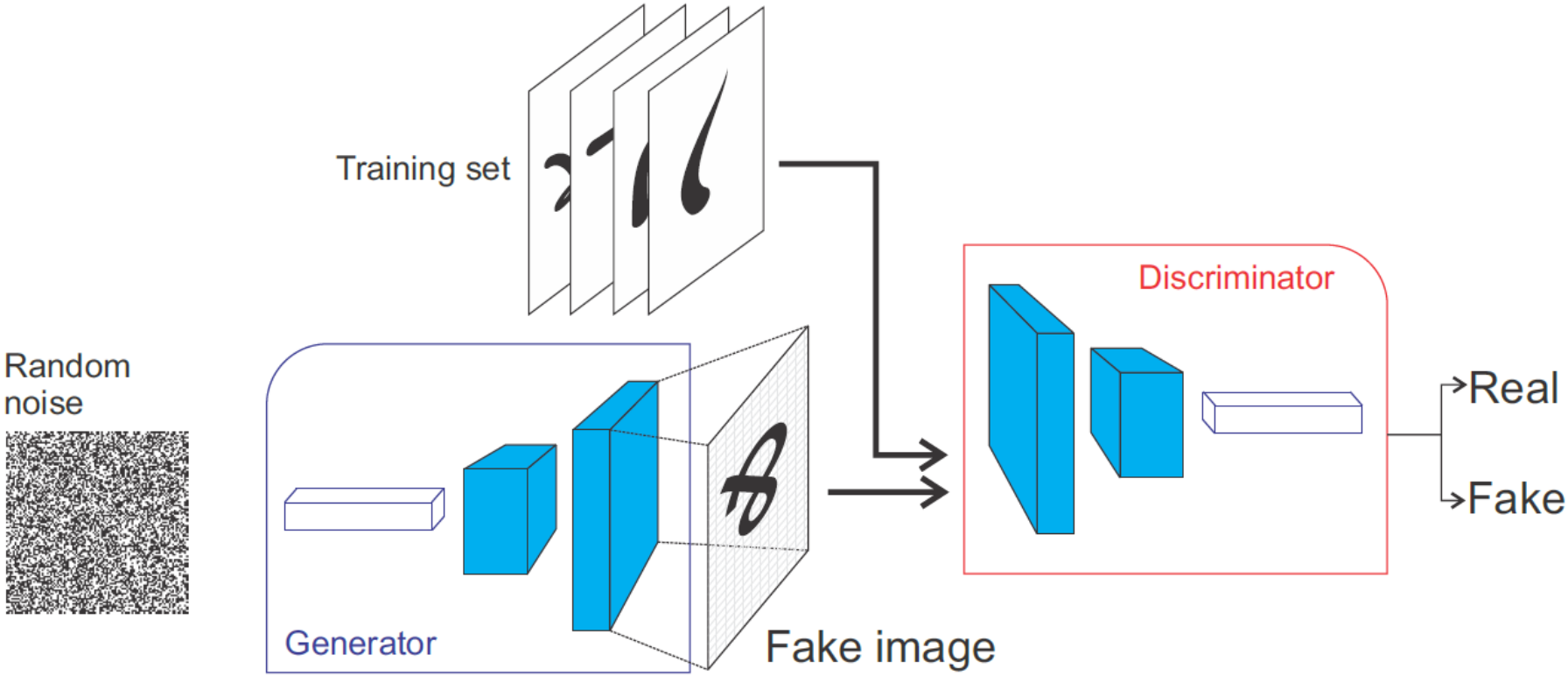
Методы улучшения сходимости GAN

Викторов Борис Викторович
M4145
Обработка и генерация изображений
Весна 2026 г.

Generative adversarial network



[Content preview from Machine Learning with Scala Quick Start Guide]

Generative adversarial network

- Дискриминатор и Генератор играют в "минимакс игру":

$$\min_G \max_D V(D,G) = \underset{x\sim p_{data}}{E}[logD(x)] + \underset{z\sim p_z}{E}[log(1-D(G(z)))]$$

- Дискриминатор обучается максимизировать:

$$\mathrm{E}_{\mathbf{x}\sim p_{\mathrm{data}}(\mathbf{x})}[\log D(\mathbf{x})] + \mathrm{E}_{\mathbf{x}\sim p_g(\mathbf{x})}[\log(1-D(\mathbf{x}))]$$

- Генератор обучается минимизировать:

$$\mathrm{E}_{\mathbf{z}\sim p_z(\mathbf{z})}[\log(1-D(G(\mathbf{z})))]$$

Goodfellow I. J. et al. Generative adversarial nets //Advances in neural information processing systems. – 2014. – Т. 27.

Non-Saturating Loss

- Замена функции ошибки для генератора с

$$\min log(1 - D)$$

на

$$\max logD$$

- Исходный вариант имеет маленький градиент на раннем этапе обучения и большой градиент при сходимости, а предложенный наоборот.

Goodfellow I. J. et al. Generative adversarial nets //Advances in neural information processing systems. – 2014. – Т. 27.

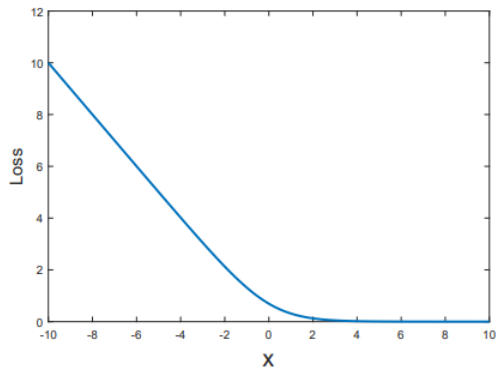
Least Squares Generative Adversarial Networks(LSGAN)

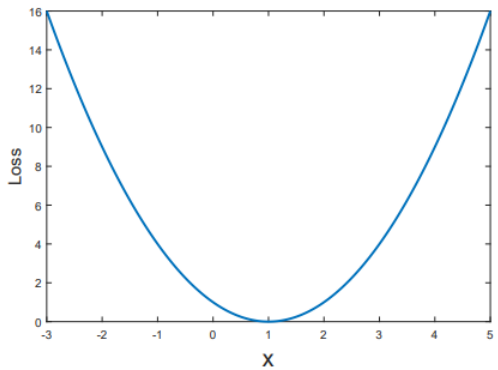
- Предлагается схема a-b для дискриминатора, где a и b - метки для фейковых данных и реальных данных соответственно. Перейдём к квадратичной функции ошибки:

$$\min_D V_{\mathrm{LSGAN}}(D) = \frac{1}{2}\mathbb{E}_{\boldsymbol{x}\sim p_{\mathrm{data}}(\boldsymbol{x})}\left[(D(\boldsymbol{x})-b)^2\right] + \frac{1}{2}\mathbb{E}_{\boldsymbol{z}\sim p_{\boldsymbol{z}}(\boldsymbol{z})}\left[(D(G(\boldsymbol{z}))-a)^2\right]$$

$$\min_G V_{\mathrm{LSGAN}}(G) = \frac{1}{2}\mathbb{E}_{\boldsymbol{z}\sim p_{\boldsymbol{z}}(\boldsymbol{z})}\left[(D(G(\boldsymbol{z}))-c)^2\right],$$

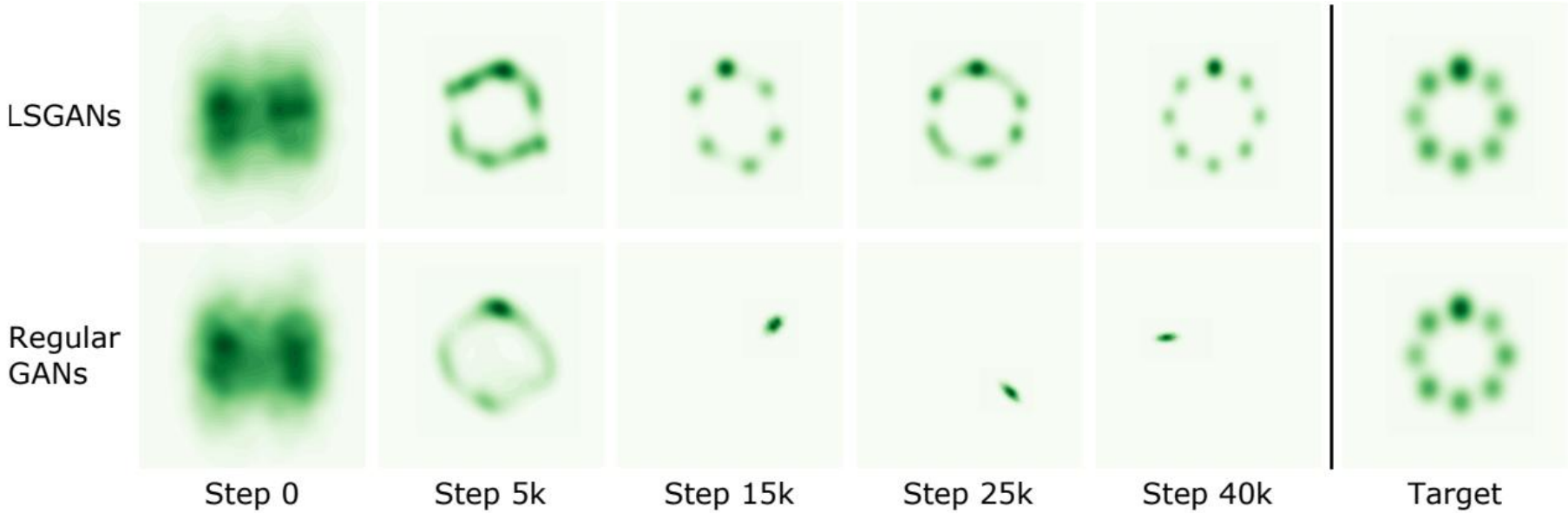
- Дискриминатор учится отвечать a для фейков и b для настоящих данных, а генератор хочет убедить его отвечать c на фейках. Обычно берут просто a = 0, b = c = 1.





Mao X. et al. Least squares generative adversarial networks //Proceedings of the IEEE international conference on computer vision. – 2017. – С. 2794-2802.

Least Squares Generative Adversarial Networks(LSGAN)



Mao X. et al. Least squares generative adversarial networks //Proceedings of the IEEE international conference on computer vision. – 2017. – С. 2794-2802.

Wasserstein GAN (WGAN)

- Давайте посмотрим на другие метрики близости между p_{data} и p_{model}.
- Earth Mover distance, или Wasserstein-1:

$$W(p_{\text{data}}, p_{\text{model}}) = \inf_{\gamma \in \Pi(p_{\text{data}}, p_{\text{model}})} \mathbb{E}_{(\mathbf{x},\mathbf{y})\sim\gamma} \left[\|\mathbf{x} - \mathbf{y}\|\right],$$

- Двойственность Монжа-Канторовича (Kantorovich-Rubinstein duality) говорит, что данная метрика эквивалетна:

$$W(p_{\text{data}}, p_{\text{model}}) = \sup_{\|f\|_L \leq 1} \left(\mathbb{E}_{\mathbf{x}\sim p_{\text{data}}} \left[f(\mathbf{x})\right] - \mathbb{E}_{\mathbf{x}\sim p_{\text{model}}} \left[f(\mathbf{x})\right]\right),$$

- где супремум берётся по всем функциям с липшицевой константой ≤ 1.

Arjovsky M., Chintala S., Bottou L. Wasserstein generative adversarial networks //International conference on machine learning. – Pmlr, 2017. – С. 214-223.

Wasserstein GAN (WGAN)

Algorithm 1 WGAN, our proposed algorithm. All experiments in the paper used the default values $\alpha = 0.00005$, $c = 0.01$, $m = 64$, $n_{\text{critic}} = 5$.

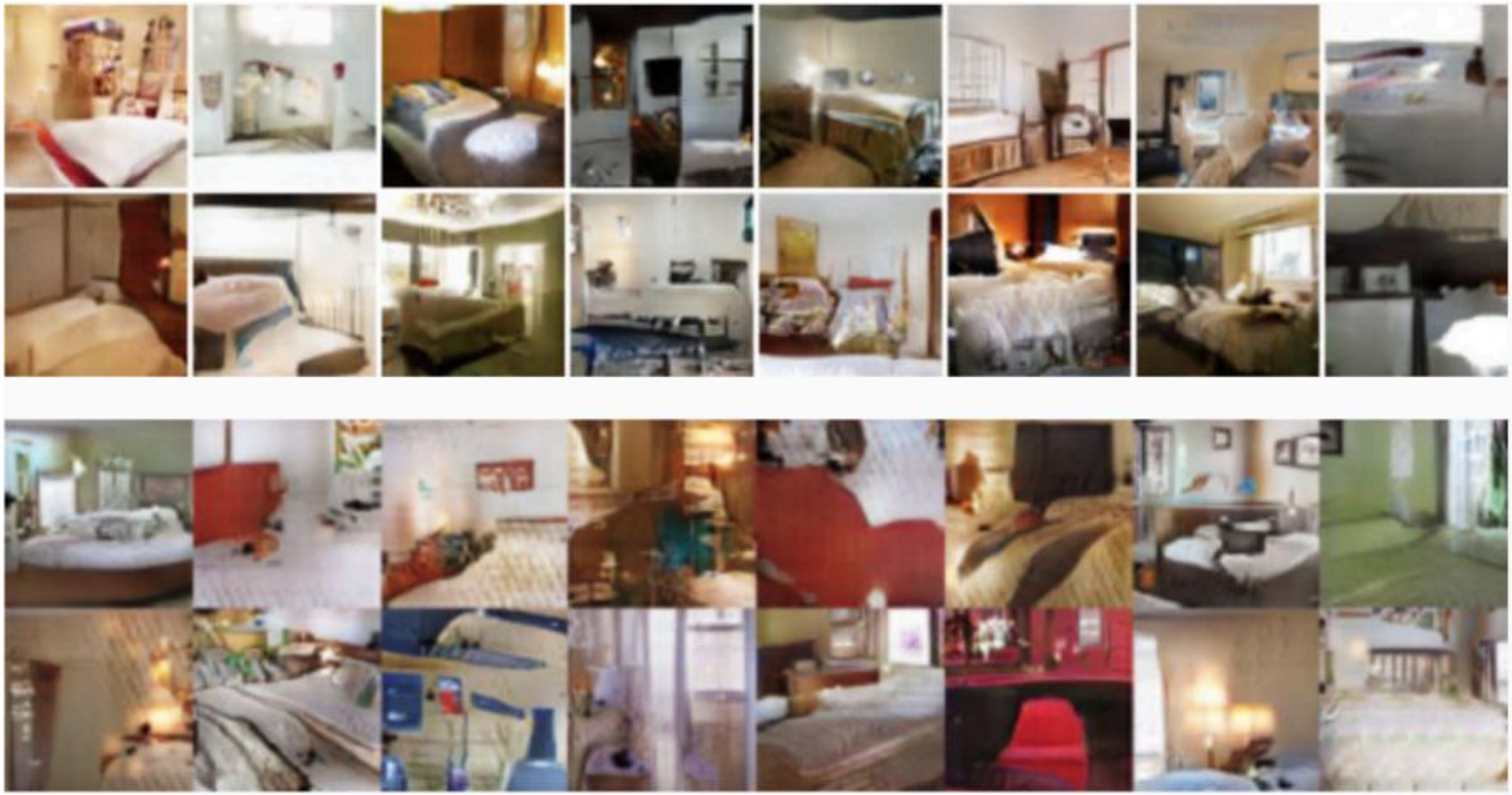
Require: : α, the learning rate. c, the clipping parameter. m, the batch size. n_{critic}, the number of iterations of the critic per generator iteration.

Require: : w_0, initial critic parameters. θ_0, initial generator's parameters.

while θ has not converged **do**
  for $t = 0, ..., n_{\text{critic}}$ **do**
    Sample $\{x^{(i)}\}_{i=1}^m \sim \mathbb{P}_r$ a batch from the real data.
    Sample $\{z^{(i)}\}_{i=1}^m \sim p(z)$ a batch of prior samples.
    $g_w \leftarrow \nabla_w \left[\frac{1}{m}\sum_{i=1}^m f_w(x^{(i)}) - \frac{1}{m}\sum_{i=1}^m f_w(g_\theta(z^{(i)}))\right]$
    $w \leftarrow w + \alpha \cdot \text{RMSProp}(w, g_w)$
    $w \leftarrow \text{clip}(w, -c, c)$
  end for
  Sample $\{z^{(i)}\}_{i=1}^m \sim p(z)$ a batch of prior samples.
  $g_\theta \leftarrow -\nabla_\theta \frac{1}{m}\sum_{i=1}^m f_w(g_\theta(z^{(i)}))$
  $\theta \leftarrow \theta - \alpha \cdot \text{RMSProp}(\theta, g_\theta)$
end while

Arjovsky M., Chintala S., Bottou L. Wasserstein generative adversarial networks //International conference on machine learning. – Pmlr, 2017. – С. 214-223.

Wasserstein GAN (WGAN)



Arjovsky M., Chintala S., Bottou L. Wasserstein generative adversarial networks //International conference on machine learning. – Pmlr, 2017. – C. 214-223.

WGAN-GP

- Weight clipping в критике на самом деле ведёт к проблемам:
  - Если clipping большой, то может потребоваться много времени, чтобы любые веса достигли своего предела.
  - Если clipping маленький, это может легко привести к исчезновению градиентов.

Лучше добавить мягкий регуляризатор на градиент:

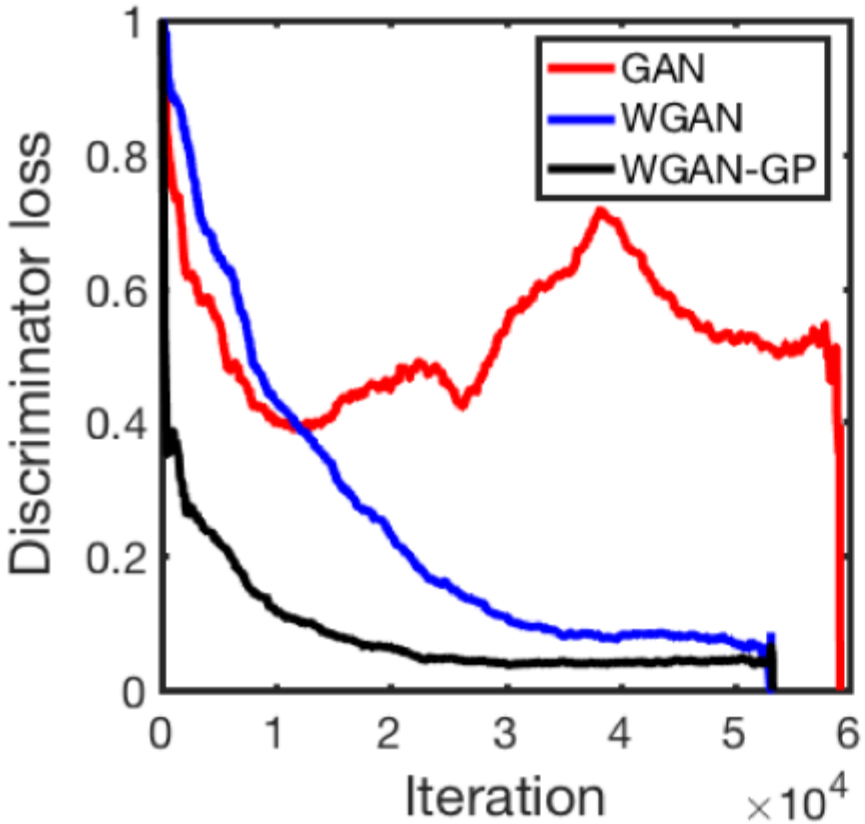
$$\lambda \mathbb{E}_{\hat{\mathbf{x}} \sim P_{\hat{\mathbf{x}}}} \left[\left(\|\nabla_{\hat{\mathbf{x}}} D(\hat{\mathbf{x}})\|_2 - 1 \right)^2 \right]$$

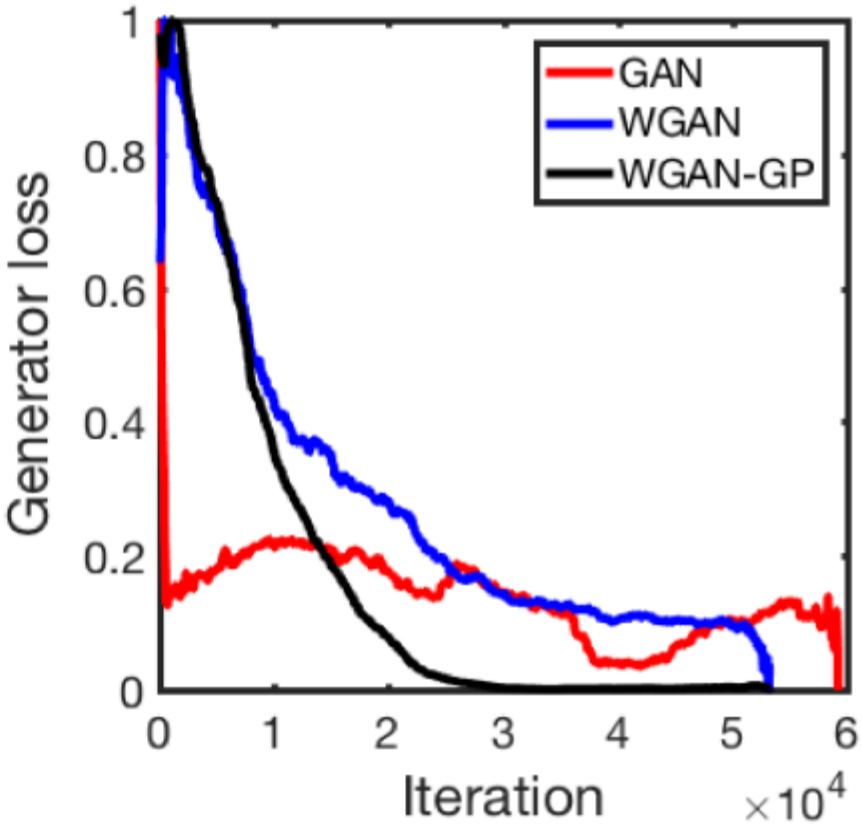
Полный лосс дискриминатора:

$$J_D = \mathop{\mathbb{E}}_{\tilde{x} \sim \mathbb{P}_g} [D(\tilde{x})] - \mathop{\mathbb{E}}_{x \sim \mathbb{P}_r} [D(x)] + \lambda \mathop{\mathbb{E}}_{\hat{x} \sim \mathbb{P}_{\hat{x}}} [\|\nabla_{\hat{x}} D(\hat{x})\|_2 - 1)^2]$$

Chen, Zhimin and Yuguang Tong. “Face Super-Resolution Through Wasserstein GANs.” *ArXiv* abs/1705.02438 (2017): n. pag.

Wasserstein GAN (WGAN)





Chen, Zhimin and Yuguang Tong. "Face Super-Resolution Through Wasserstein GANs." *ArXiv* abs/1705.02438 (2017): n. pag.

Spectral Normalization GAN (SN-GAN)

- Применяется спектральная нормализация к весам дискриминатора для ограничения липшицевой константой ≤ 1.
- Спектральная норма: $\sigma(A) := \max_{\boldsymbol{h}:\boldsymbol{h}\neq\boldsymbol{0}} \frac{\|A\boldsymbol{h}\|_2}{\|\boldsymbol{h}\|_2} = \max_{\|\boldsymbol{h}\|_2\leq 1} \|A\boldsymbol{h}\|_2,$
- Линейный слой можно записать $g(\boldsymbol{h}) = W\boldsymbol{h}$, тогда

$$\|g\|_{\mathrm{Lip}} = \sup_{\boldsymbol{h}} \sigma(\nabla g(\boldsymbol{h})) = \sup_{\boldsymbol{h}} \sigma(W) = \sigma(W)$$

- Тогда получим границу

$$\|f\|_{\mathrm{Lip}} \leq \|(\boldsymbol{h}_L \mapsto W^{L+1}\boldsymbol{h}_L)\|_{\mathrm{Lip}} \cdot \|a_L\|_{\mathrm{Lip}} \cdot \|(\boldsymbol{h}_{L-1} \mapsto W^{L}\boldsymbol{h}_{L-1})\|_{\mathrm{Lip}}$$

$$\cdots \|a_1\|_{\mathrm{Lip}} \cdot \|(\boldsymbol{h}_0 \mapsto W^{1}\boldsymbol{h}_0)\|_{\mathrm{Lip}} = \prod_{l=1}^{L+1} \|(\boldsymbol{h}_{l-1} \mapsto W^{l}\boldsymbol{h}_{l-1})\|_{\mathrm{Lip}} = \prod_{l=1}^{L+1} \sigma(W^l)$$

- Нормализуем веса $\bar{W}_{\mathrm{SN}}(W) := W/\sigma(W)$. Тогда получим $\sigma\left(\bar{W}_{\mathrm{SN}}(W)\right) = 1$ что и даст нужное нам ограничение.

[Miyato T. et al. Spectral normalization for generative adversarial networks //arXiv preprint arXiv:1802.05957. – 2018.]

Spectral Normalization GAN (SN-GAN)

Algorithm 1 SGD with spectral normalization

- Initialize $\tilde{\boldsymbol{u}}_l \in \mathcal{R}^{d_l}$ for $l = 1, \ldots, L$ with a random vector (sampled from isotropic distribution).
- For each update and each layer l:
  1. Apply power iteration method to a unnormalized weight W^l:

$$\tilde{\boldsymbol{v}}_l \leftarrow (W^l)^{\mathrm{T}} \tilde{\boldsymbol{u}}_l / \|(W^l)^{\mathrm{T}} \tilde{\boldsymbol{u}}_l\|_2 \tag{20}$$

$$\tilde{\boldsymbol{u}}_l \leftarrow W^l \tilde{\boldsymbol{v}}_l / \|W^l \tilde{\boldsymbol{v}}_l\|_2 \tag{21}$$

  2. Calculate $\bar{W}_{\mathrm{SN}}$ with the spectral norm:

$$\bar{W}^l_{\mathrm{SN}}(W^l) = W^l / \sigma(W^l), \text{ where } \sigma(W^l) = \tilde{\boldsymbol{u}}_l^{\mathrm{T}} W^l \tilde{\boldsymbol{v}}_l \tag{22}$$

  3. Update W^l with SGD on mini-batch dataset $\mathcal{D}_M$ with a learning rate α:

$$W^l \leftarrow W^l - \alpha \nabla_{W^l} \ell(\bar{W}^l_{\mathrm{SN}}(W^l), \mathcal{D}_M) \tag{23}$$

Miyato T. et al. Spectral normalization for generative adversarial networks //arXiv preprint arXiv:1802.05957. – 2018.

Progressive Growing of GANs (PGGAN)

- Создание изображений с высоким разрешением затруднено:
  - легче отличить сгенерированные изображения от обучающих, что значительно усугубляет проблему градиента.
  - требует использования batch меньшего размера из-за нехватки памяти.

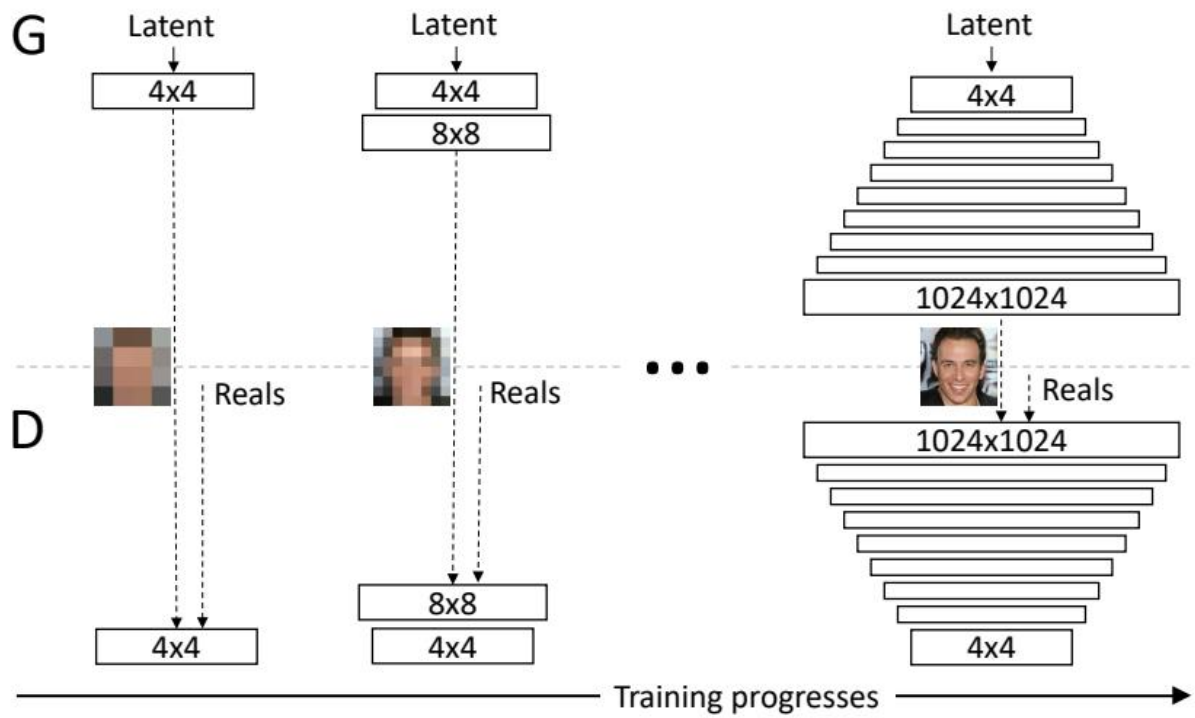
- Основная идея заключается в том, чтобы постепенно расширять возможности как генератора, так и дискриминатора, начиная с более простых изображений с низким разрешением и добавляя новые слои, которые по мере обучения будут отображать детали с более высоким разрешением.

- Это значительно ускоряет обучение и повышает стабильность работы в высоком разрешении

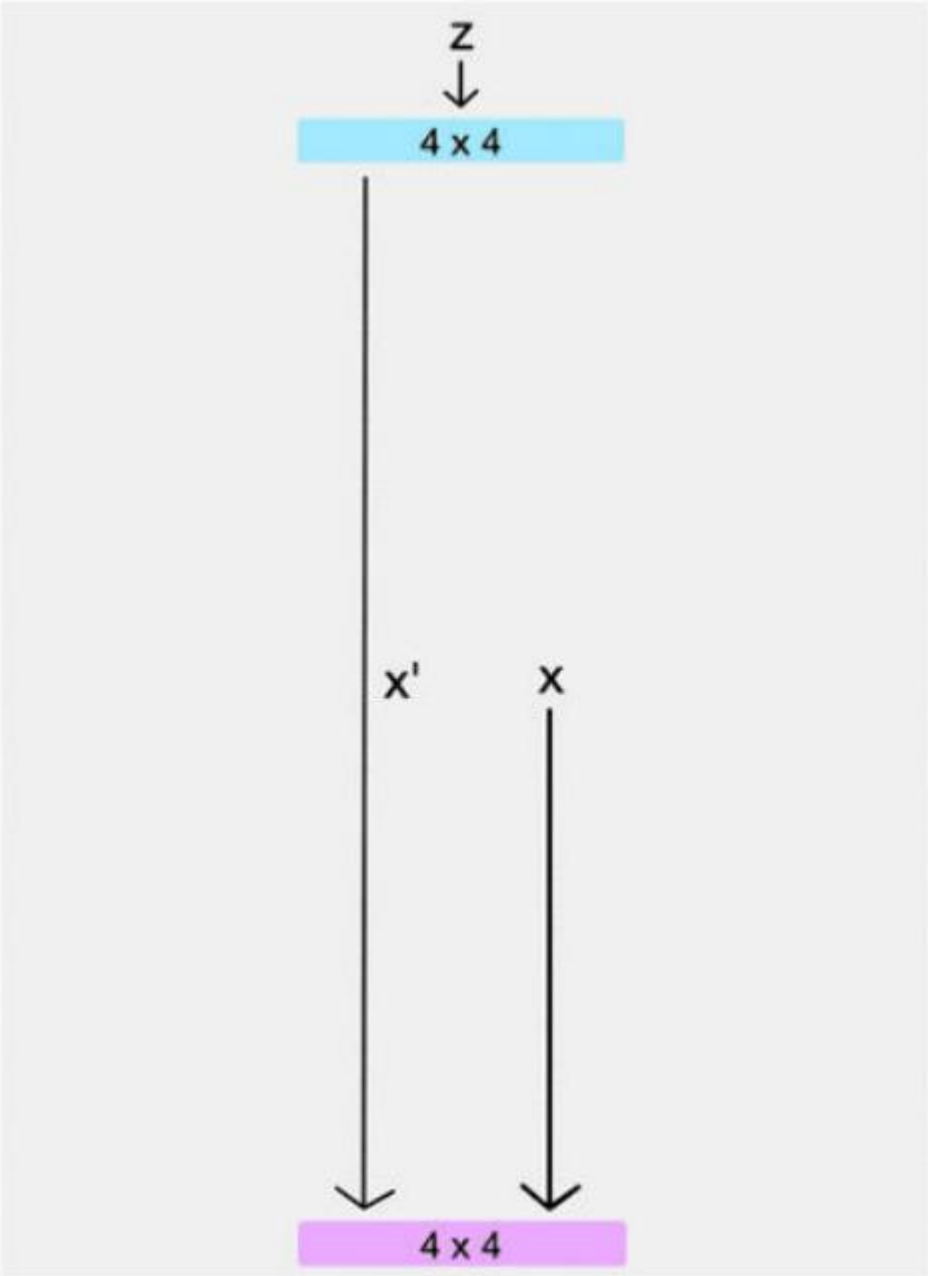
Karras T. et al. Progressive growing of gans for improved quality, stability, and variation //arXiv preprint arXiv:1710.10196. – 2017.

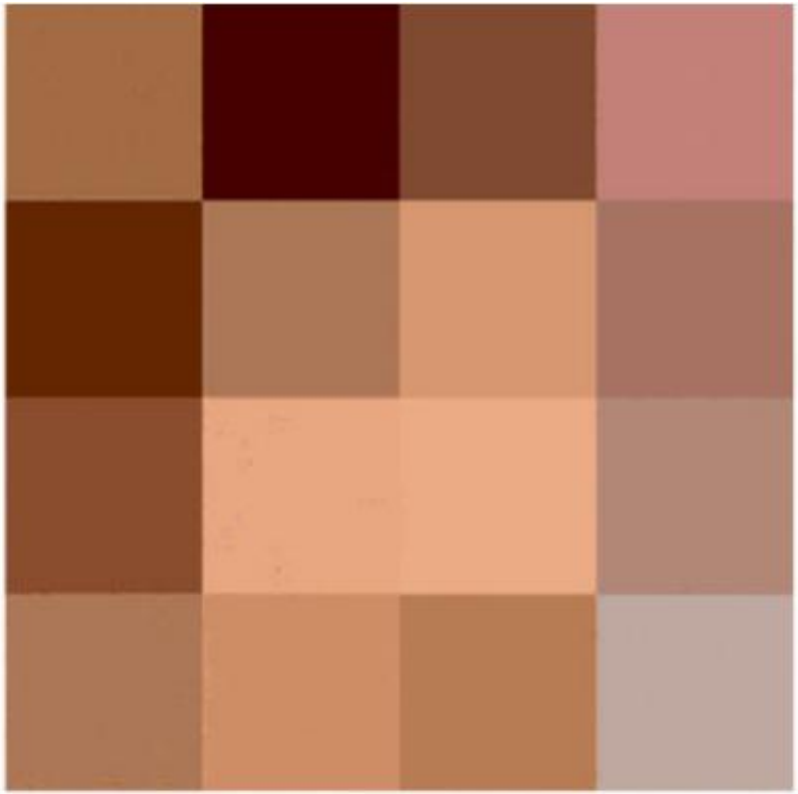
Progressive Growing of GANs (PGGAN)

- На каждом этапе обучаем GAN создавать картинки вчетверо больше, начиная от 4 × 4 и так постепенно увеличивают размерность до 1024 × 1024.



[Karras T. et al. Progressive growing of gans for improved quality, stability, and variation //arXiv preprint arXiv:1710.10196. – 2017.]





Training time: 0 days

4x4 resolution

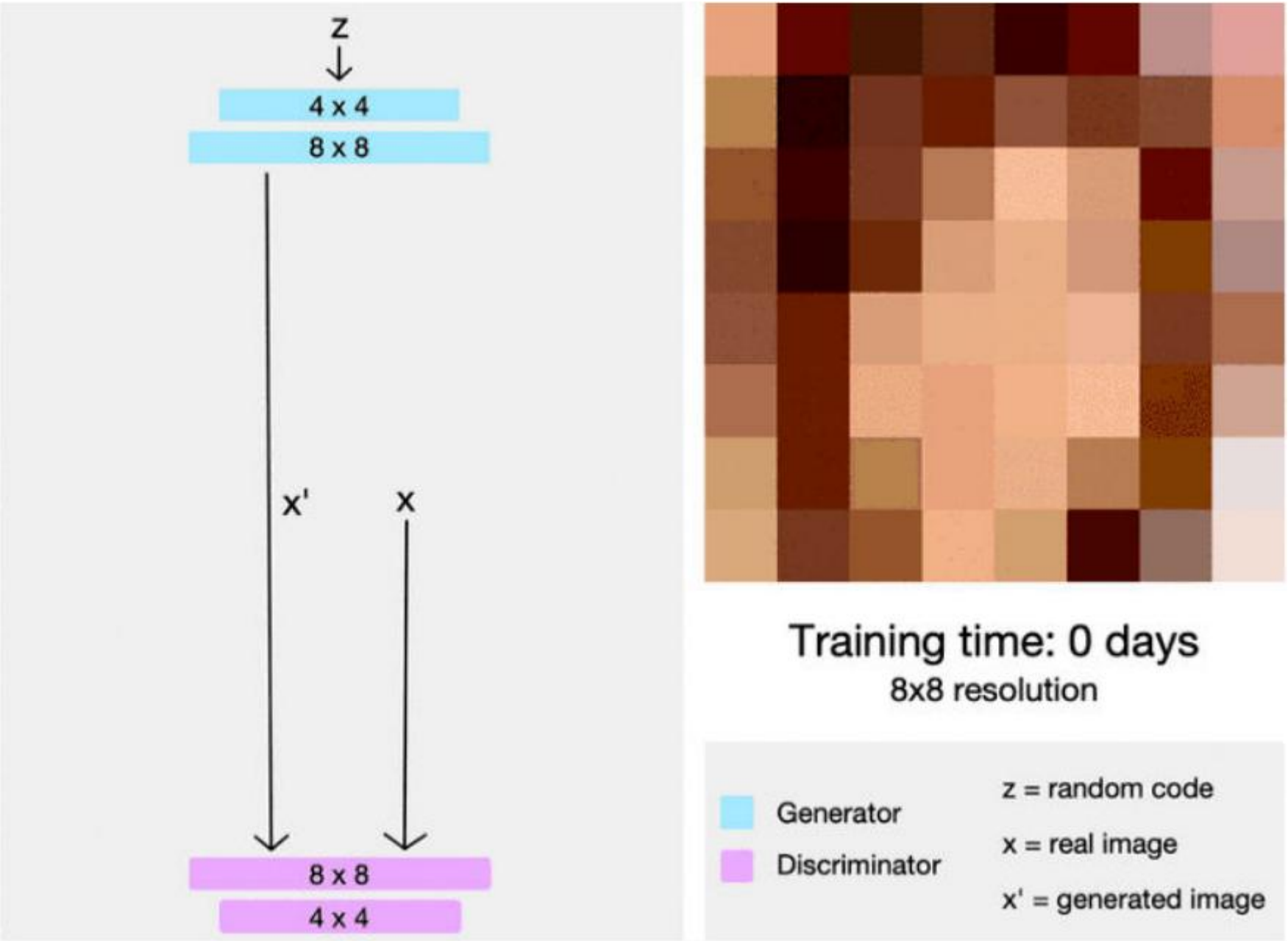
Generator

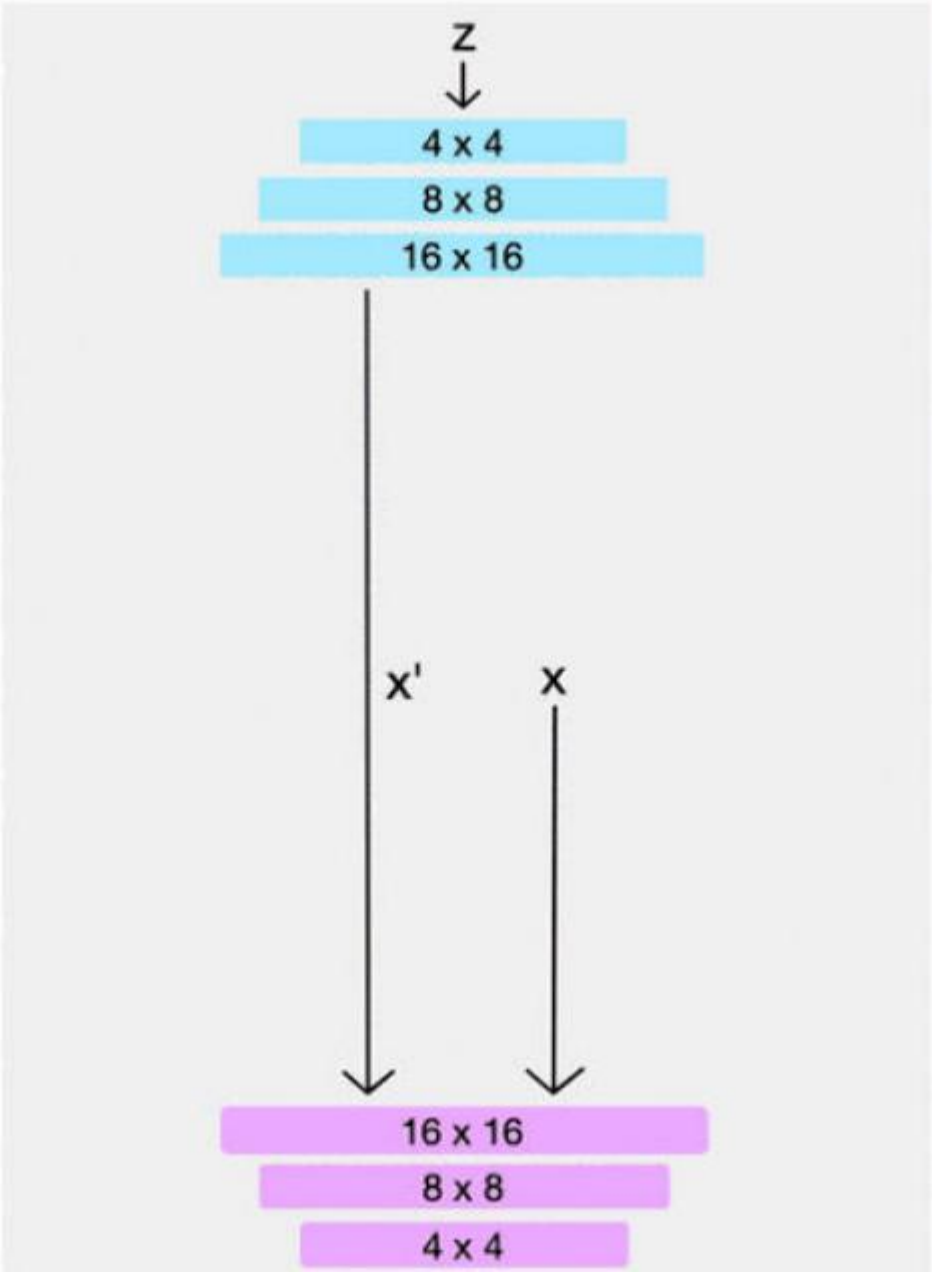
Discriminator

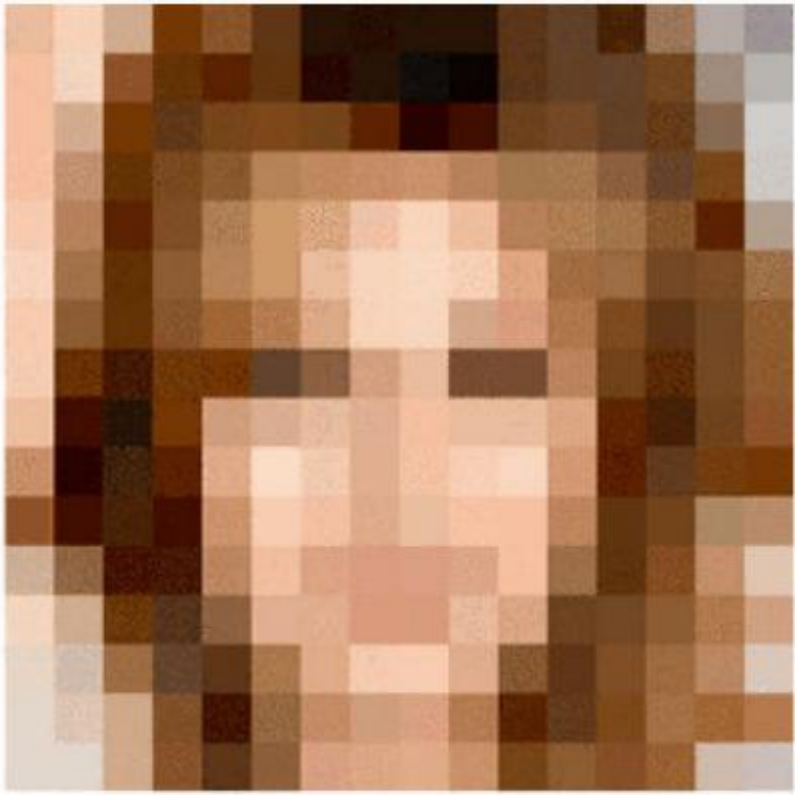
z = random code

x = real image

x' = generated image


z
4 x 4
8 x 8
x'
x
8 x 8
4 x 4
Training time: 0 days
8x8 resolution
Generator
Discriminator
z = random code
x = real image
x' = generated image





Training time: 0 days

16x16 resolution

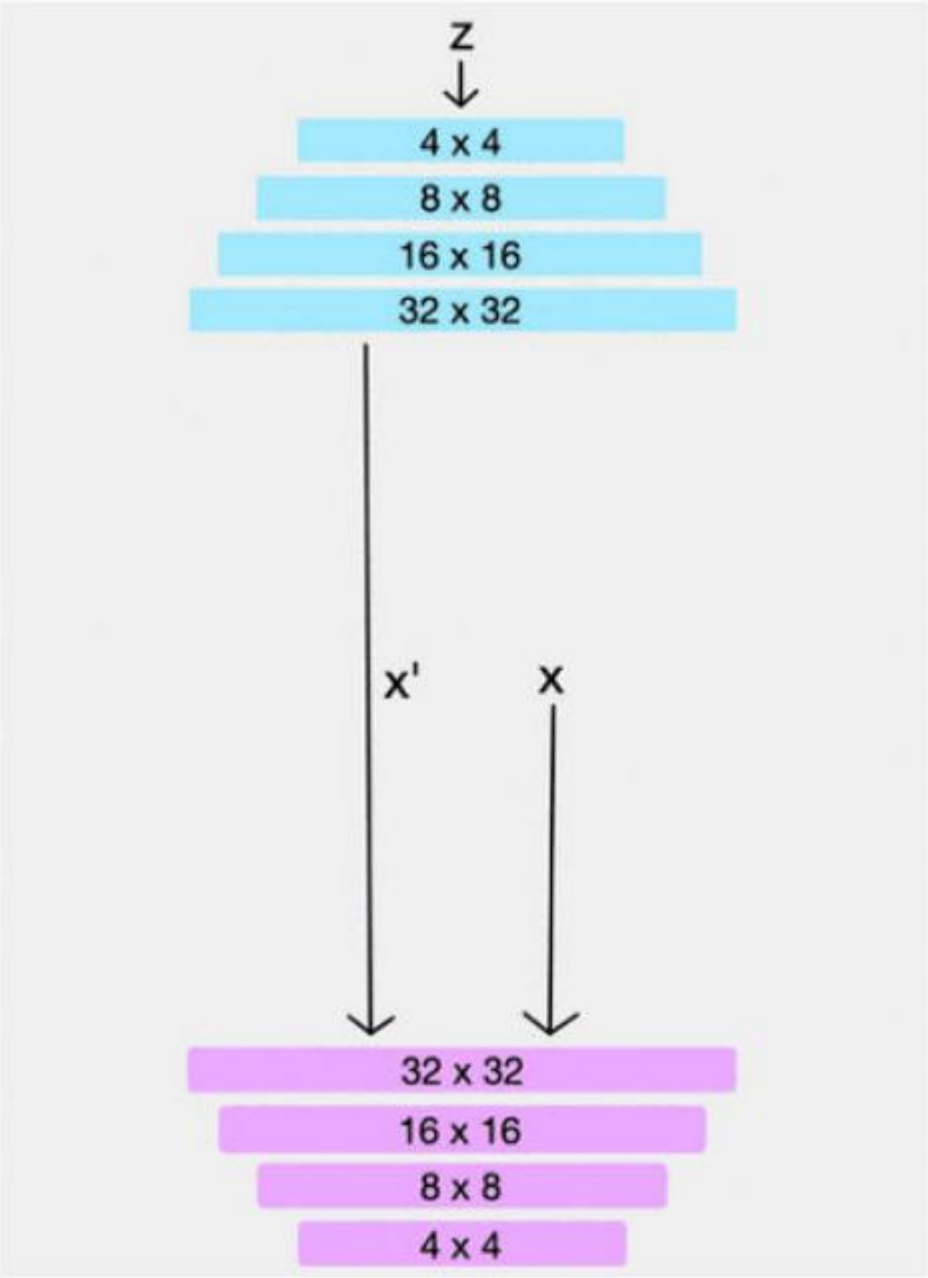
Generator

Discriminator

z = random code

x = real image

x' = generated image



Training time: 1 day

32x32 resolution

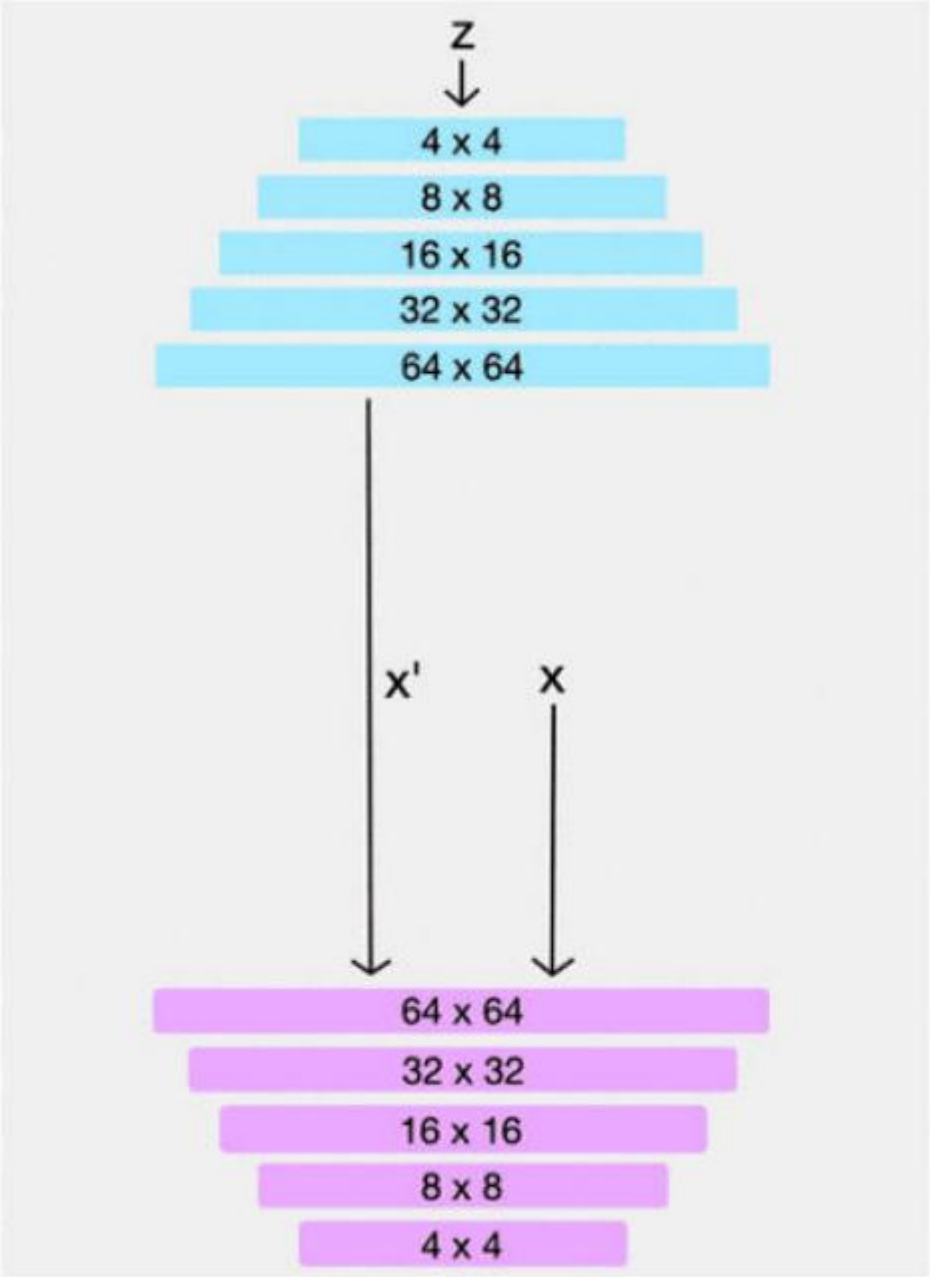
Generator

Discriminator

z = random code

x = real image

x' = generated image



Training time: 2 days

64x64 resolution

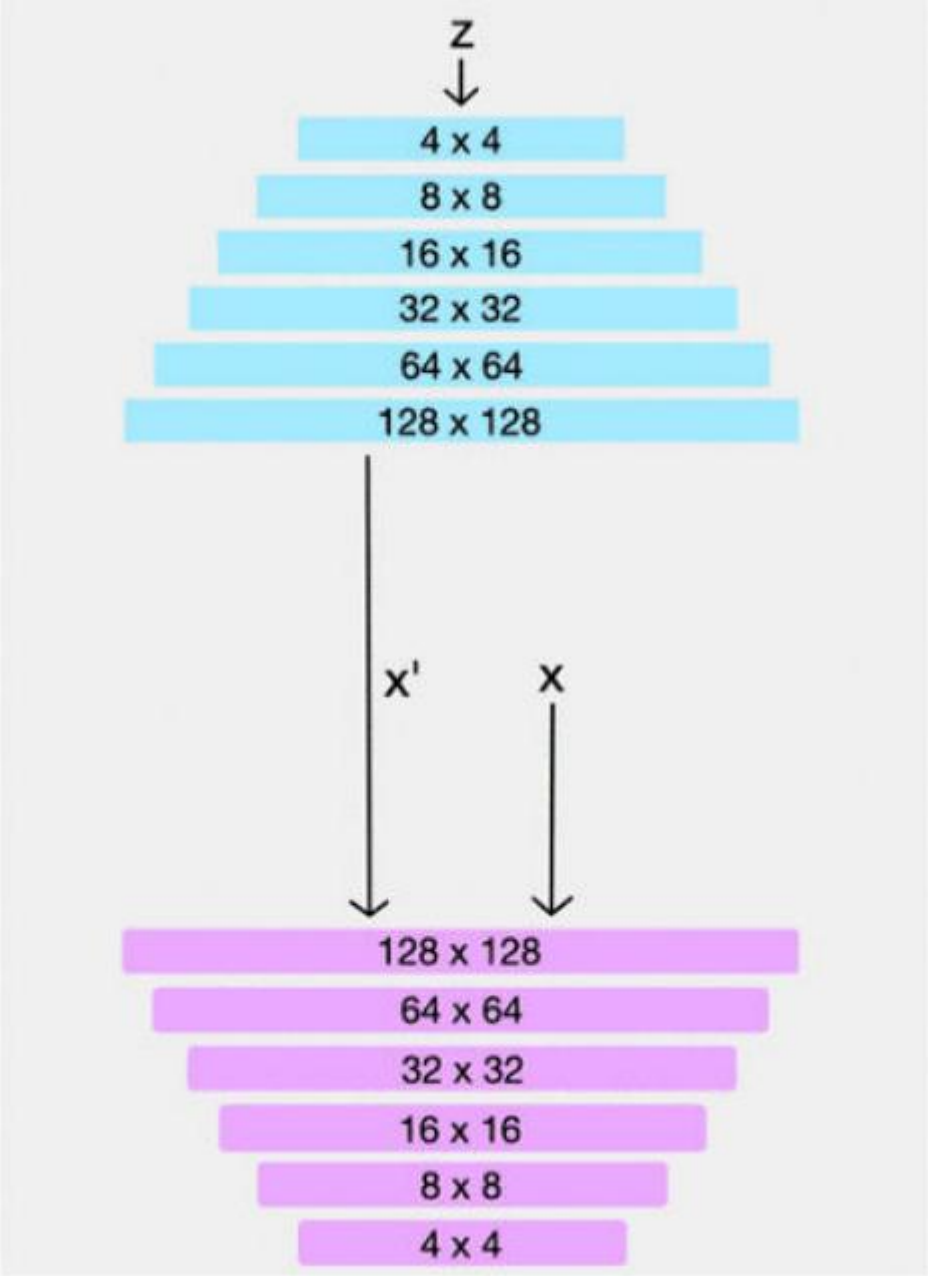
Generator

Discriminator

z = random code

x = real image

x' = generated image



Training time: 4 days

128x128 resolution

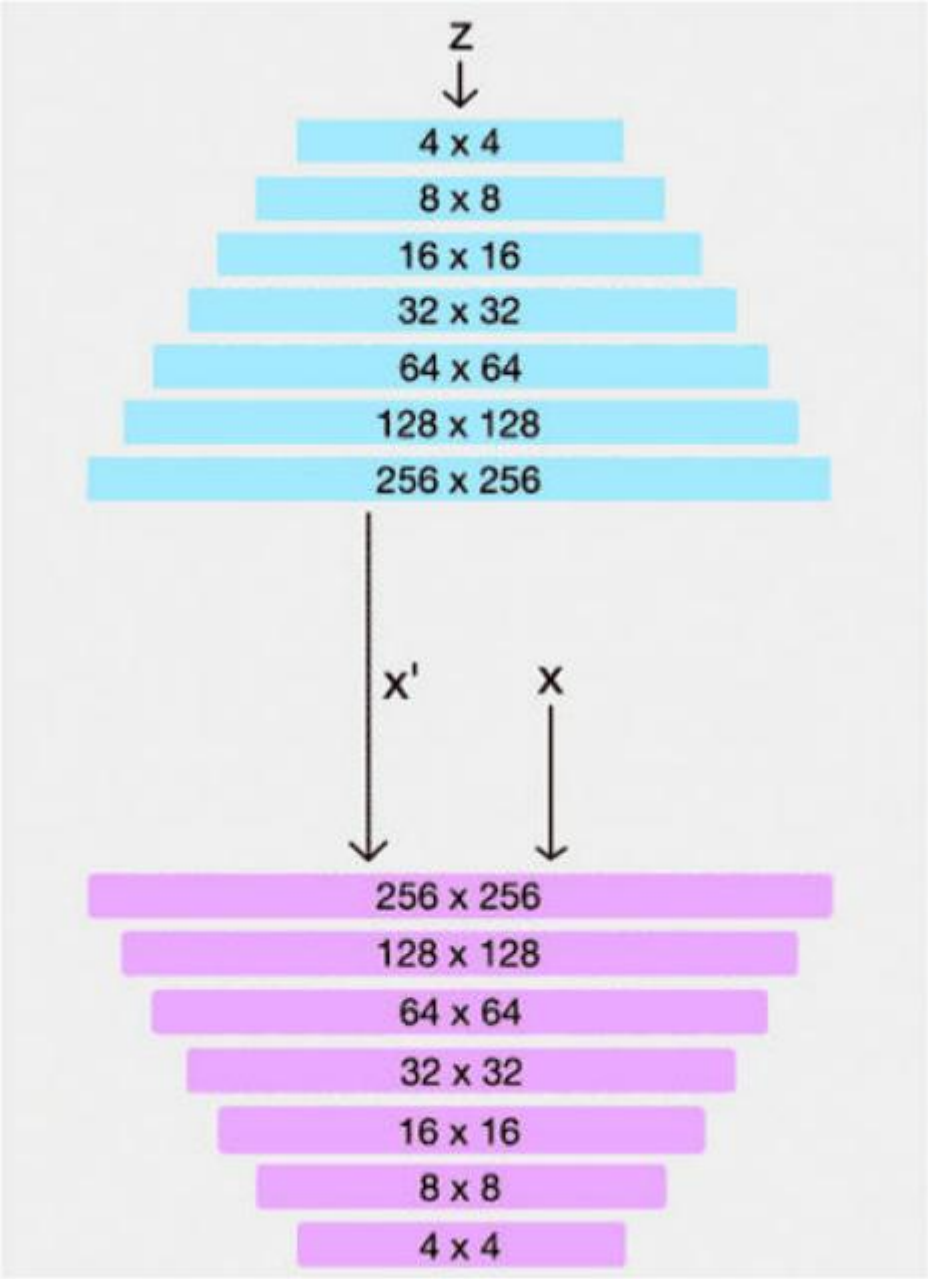
Generator

Discriminator

z = random code

x = real image

x' = generated image



Training time: 7 days

256x256 resolution

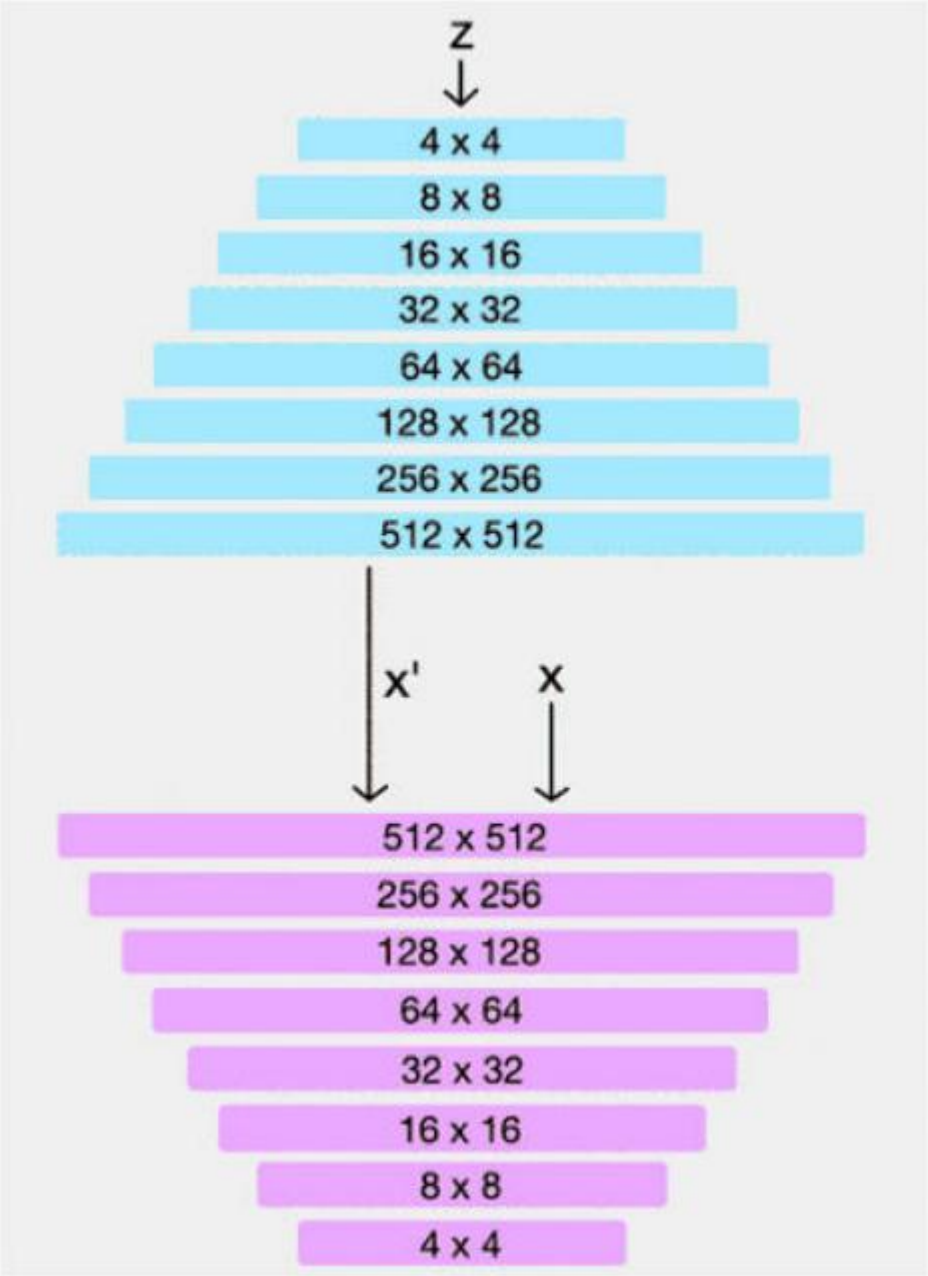
Generator

Discriminator

z = random code

x = real image

x' = generated image



Training time: 11 days

512x512 resolution

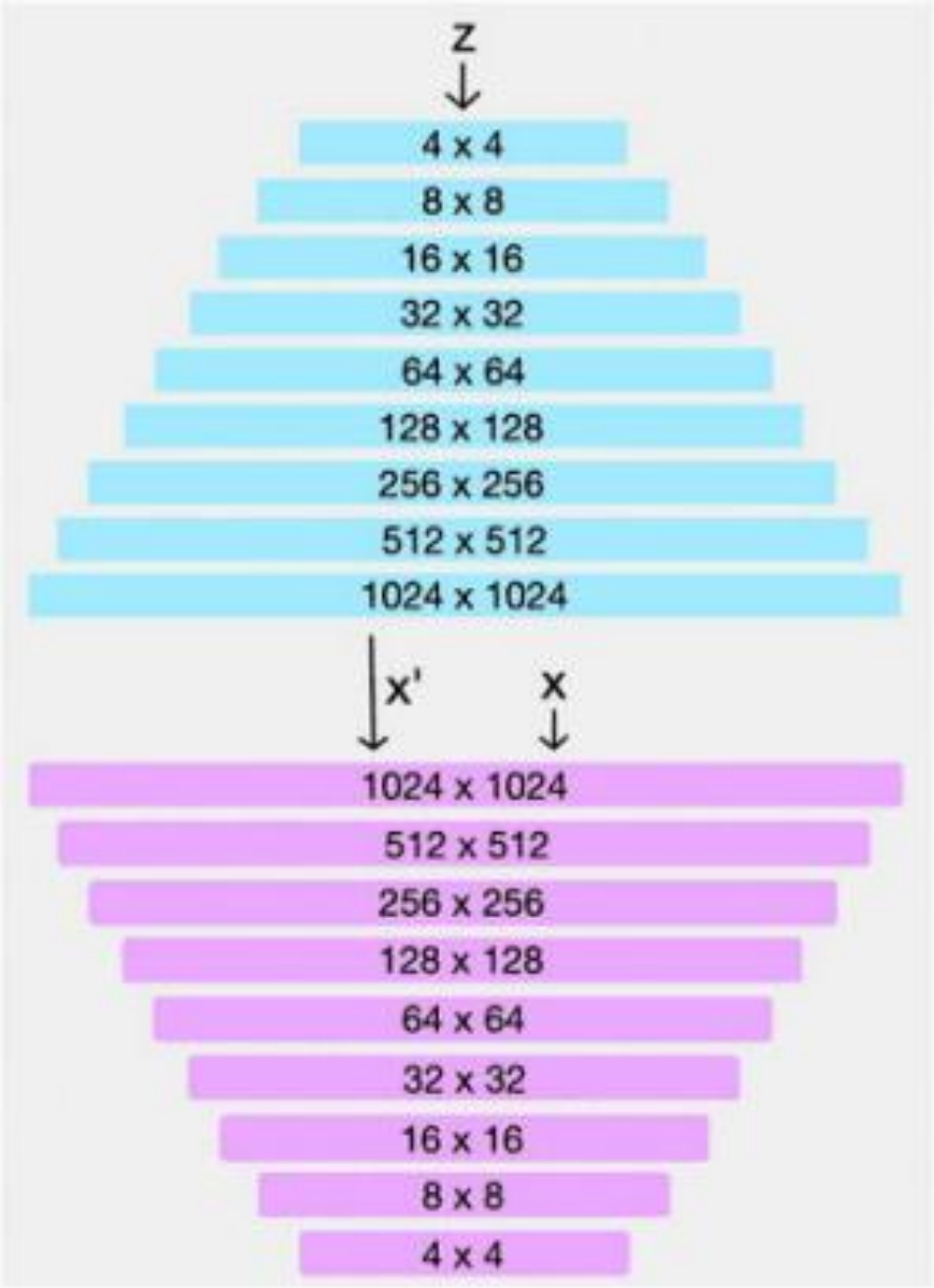
Generator

Discriminator

z = random code

x = real image

x' = generated image



Training time: 14 days

1024x1024 resolution

Generator

Discriminator

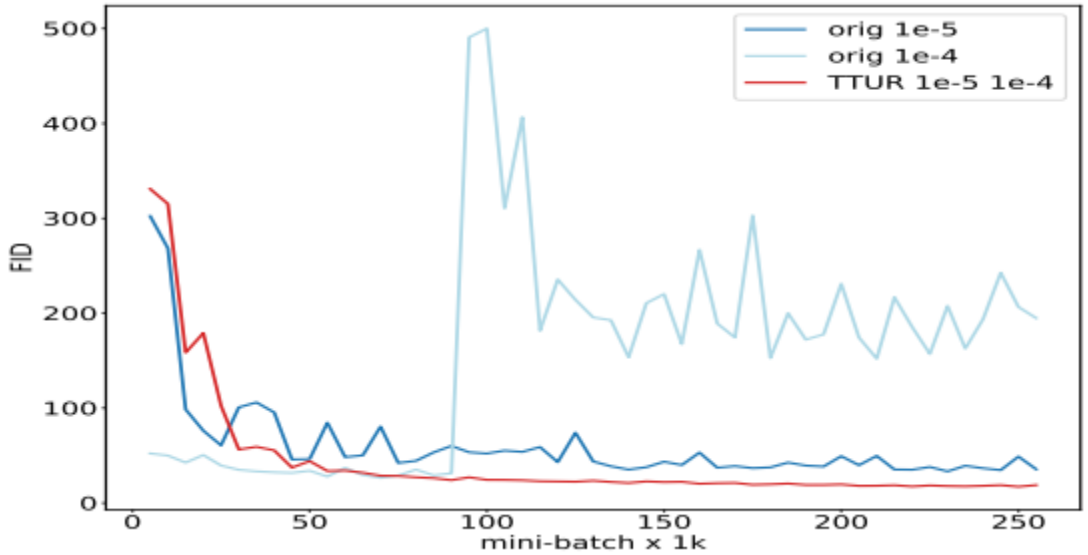
z = random code

x = real image

x' = generated image

Two Time-Scale Update Rule (TTUR)

- Предлагается сделать сходимость GAN стабильнее за счет снижения скорости обучения генератора по сравнению с дискриминатором. В статье доказывается, что при таком обучении GAN "сходятся, при умеренных предположениях, к стационарному локальному равновесию Нэша".
- Показывается, что это свойство распространяется и на использование оптимизатора Adam.



[Heusel M. et al. Gans trained by a two time-scale update rule converge to a local nash equilibrium //Advances in neural information processing systems. – 2017. – T. 30.]

Спасибо за внимание!

- Хочется сделать ссылку на лекцию Сергея Игоревича Николенко “Порождающие состязательные сети”, откуда была взята часть материала для данного доклада.
- Вкладка с лекциями: https://www.sergeynikolenko.ru/en/courses/mlspsu2024-3
- Слайды лекции: https://cms.sergeynikolenko.ru/uploads/38_gan2_compr_10adfd0f72.pdf